

Příprava dat

Čištění a transformace dat pro modelování

Vojtěch Ondryhal

K209, FVT, UO

LS 2026, Analýza informačních zdrojů

- 1 Přechod od diagnostiky k sanaci dat
- 2 Taxonomie datové kontaminace (dirty data)
- 3 Teorie chybějících hodnot
- 4 Škálování a transformace příznaků
- 5 Ošetření odlehlých pozorování
- 6 Redukce dimenzionality a lineární transformace
- 7 Redukce početnosti: Teorie vzorkování
- 8 Transformace a inženýrství příznaků

Přechod od diagnostiky k sanaci dat

Přechod od diagnostiky k sanaci dat

- Předchozí fáze (Univariační a Bivariační EDA) sloužila k identifikaci strukturálních nedostatků v datovém souboru.
- Následující kroky představují technickou sanaci dat před vstupem do analytických modelů.

Identifikované artefakty (EDA)

- ⚠ **Odlehlá pozorování:** Hodnoty deformující odhady parametrů ($\bar{x}$, σ^2).
- ⚠ **Chybějící data:** Absence záznamů snižující robustnost vzorku.
- ⚠ **Heterogenita měřítek:** Výrazné rozdíly v řádech proměnných.
- ⚠ **Nesymetrie distribucí:** Extrémní šikmost vyžadující stabilizaci rozptylu.



Technická příprava (Preprocessing)

- ✓ **Data Cleaning:** Filtrace nebo korekce anomálních pozorování.
- ✓ **Data Imputation:** Statistické doplnění chybějících hodnot.
- ✓ **Feature Scaling:** Standardizace a normalizace vektorů.
- ✓ **Power Transformations:** Funkcionální úprava tvaru rozdělení.

„EDA nám řekla, co je špatně. Příprava dat je proces, jak to opravit, než začneme modelovat.“

Co je příprava dat? (Scrubbing)

Proces detekce, opravy a filtrace nekonzistentních, chybných či neúplných záznamů s cílem transformovat surová data do validní podoby pro analýzu a modelování.

- **Časová náročnost:** Nejdražší fáze celého projektu (80 % času).
- **Cíl:** Přejít ze surového stavu do datové matice $\mathbf{X}$.

Ukotvení v procesním cyklu

Žádný model nefunguje izolovaně. Příprava dat tvoří jádro standardních frameworků:

- **CRISP-DM:** Fáze *Data Preparation* (následuje po porozumění datům a předchází modelování).
- **OSEMN:** Fáze *Scrub* (druhý krok, úzce propojený s fází *Obtain* a *Explore*).



Přehled technik: Co nás čeká?

1. Sanace a čištění anomálií

Oprava a eliminace nedostatků v datech:

- **Detekce a taxonomie:** Rozlišování deterministických defektů, šumu a systematického biasu.
- **Chybějící hodnoty:** Strategie odstranění vs. imputace (střední hodnota, k-NN, predikční modely).
- **Extrémní hodnoty:** Identifikace odlehlých měření (outliers) a jejich ošetření.

2. Transformace a vzorkování

Příprava prostoru pro matematické modely:

- **Normalizace a škálování:** Sjednocení měřítek (Min-Max, Z-Score vs. Robust).
- **Vzorkování dat:** Strategie podvýběru (s vrácením vs. bez vrácení).
- **Kódování a redukce:** Příprava kategoriálních proměnných do numerické podoby.

Taxonomie datové kontaminace (dirty data)

Sledujeme generativní proces vzniku dat s cílem identifikovat anomální jevy:

1. Deterministické defekty

Porušení integritních omezení databázových systémů (např. záporný věk, syntaktické anomálie v textových řetězcích).

2. Stochastické chyby (Šum)

Náhodné variace ε s nulovou střední hodnotou ($\mathbb{E}[\varepsilon] = 0$), způsobené limitami měřicích přístrojů.

3. Systematické zkreslení (Bias)

Nenáhodná odchylka v distribuci, často způsobená chybným designem experimentu či výběrovým kognitivním zkreslením.

Charakteristika

Chyby vykazující stoprocentní a jednoznačné porušení pevných logických, syntaktických nebo systémových pravidel. Nejedná se o pravděpodobnostní jev.

Typické příčiny:

- Chyby v integrační logice (ETL)
- Lidský faktor při ručním zadávání
- Nekonzistentní datové schémata

Příklady v praxi:

- **Porušení domény:** Věk = -5 nebo Teplota $< -273.15^{\circ}\text{C}$
- **Integritní omezení:** Duplicita v primárním klíči, NULL v NOT NULL sloupci.
- **Syntaktické anomálie:** Neplatný formát e-mailu (*chybějící @*), text v číselném poli.

Metoda nápravy

Rule-based validate: Použití regulárních výrazů (Regex), databázových constraints a assert podmínek. Automatické čištění či zahazování záznamů.

Stochastické chyby (Šum)

Definice

Náhodné variace ε , u kterých se předpokládá nulová střední hodnota:

$$\mathbb{E}[\varepsilon] = 0$$

Chyby střídavě nadhodnocují a podhodnocují reálný stav, v průměru se vyruší.

Vlastnosti šumu:

- Je přirozenou součástí každého měření.
- Zvyšuje rozptyl (σ^2) odhadů, ale neposouvá jejich střední hodnotu.

Příklady v praxi:

- **Limity přístrojů:** Fluktuace digitální váhy kolem reálné hmotnosti (± 1 g).
- **Zaokrouhlování:** Kvantizační šum při digitalizaci spojitého signálu.
- **Přenosové šumy:** Náhodné výpadky v telemetrických datech.

Metoda nápravy

Statistická filtrace: Navýšení objemu dat (zákon velkých čísel), klouzavé průměry, Gaussovské vyhlazování nebo Low-pass filtry.

3. Systematické zkreslení (Bias)

Definice

Nenáhodná, jednostranná odchylka v distribuci dat. Střední hodnota chyb je nenulová:

$$\mathbb{E}[\varepsilon] \neq 0$$

Data jsou systematicky tažena jedním směrem. Větší objem dat tento problém **neřeší**.

Klíčové nebezpečí:

- Modely natrénované na těchto datech budou replikovat a fixovat tuto systematickou chybu.

Příklady v praxi:

- **Technický bias:** Špatně kalibrovaný senzor, který vždy měří o 2°C více.
- **Výběrový efekt:** Sběr dat na nereprezentativním vzorku populace (Selection bias).
- **Kognitivní zkreslení:** Záměrné zkreslování odpovědí v dotaznících (Social desirability).

Metoda nápravy

Změna metodiky: Rekalibrace hardwaru, propensity score matching, matematická korekce (odpočítání známého posunu).

Teorie chybějících hodnot

Teorie chybějících hodnot: Rubinova taxonomie

Mějme kompletní data $\mathbf{Y}$ rozdělená na pozorovanou (observed) část $\mathbf{Y}_{obs}$ a chybějící (missing) část $\mathbf{Y}_{mis}$. Definujeme indikátorovou matici chybění $\mathbf{M}$, kde $M_{ij} = 1$ pokud Y_{ij} chybí, jinak 0.

Mechanismus chybění modelujeme podmíněnou pravděpodobností $P(\mathbf{M} \mid \mathbf{Y}, \psi)$, kde ψ reprezentuje latentní parametry chybění (skrytý mechanismus vzniku chyb).

1. Missing Completely at Random (MCAR)

Chybění je zcela náhodné a nezávislé na hodnotách dat (pozorovaných i nepozorovaných).

$$P(\mathbf{M} \mid \mathbf{Y}, \psi) = P(\mathbf{M} \mid \psi)$$

Příklady (Čistá náhoda):

- *Fyzický výpadek*: Náhodný hardwarový výpadek senzoru.
- *Ztráta paketů*: Náhodný výpadek rádiového přenosu či ztráta síťového paketu při přenosu telemetrie z bezpilotního prostředku (UAV) kvůli atmosférickému šumu.

Šance, že hodnota v databázi chybí, je pro všechny řádky i situace konstantní. Nezávisí na tom, zda senzor zrovna měřil kritické maximum, nebo běžný stav.

2. Missing at Random (MAR)

Důvod chybění dat je zřejmý z jiných sloupců tabulky. Jakmile data rozdělíme do podskupin podle známých údajů, chybění se v nich stává náhodným.

$$P(\mathbf{M} \mid \mathbf{Y}, \psi) = P(\mathbf{M} \mid \mathbf{Y}_{obs}, \psi)$$

Příklady (Vazba na jiné sloupce):

- *Pohlaví* → *Příjem*: Muži v dotazníku méně často odpovídají na otázky ohledně příjmu.
- *Věk* → *Hmotnost*: Mladí lidé méně často uvádějí svou hmotnost.

Defekt v datech dokážeme algoritmicky napravit a dopočítat, protože klíč k řešení leží v pozorovaných datech ($\mathbf{Y}_{obs}$).

3. Missing Not at Random (MNAR)

Důvod chybění je skryt přímo v hodnotě, která v tabulce chybí. Ostatní sloupce nám nepomohou – mechanismus nelze ignorovat.

$$P(\mathbf{M} \mid \mathbf{Y}, \psi) = P(\mathbf{M} \mid \mathbf{Y}_{obs}, \mathbf{Y}_{mis}, \psi)$$

Příklady (Vazba na chybějící hodnotu):

- *Vysoká částka* → *Skrytý příjem*: Subjekty s extrémně vysokým příjmem jej úmyslně neuvádějí ze strachu ze závisti.
- *Vysoká váha* → *Skrytá hmotnost*: Lidé s velmi vysokou hmotností ji v dotazníku záměrně vynechávají ze studu.

Riziko zkreslení:

- Hodnota chybí právě proto, jaká je.
- Z ostatních sloupců tabulky to nemáme šanci poznat. Vzniká nebezpečné, skryté zkreslení (Bias), které běžné matematické imputace totálně znehodnotí.

Littleův MCAR test

Globální statistický test ověřující nulovou hypotézu: H_0 : **Data chybí zcela náhodně (MCAR)**.

Princip: Analogie k χ^2 testu dobré shody

- Porovnáváme pozorované průměry podskupin s očekávaným modelem:

$$d^2 = \sum_{k=1}^K \text{váha}_k \cdot \frac{\overbrace{\bar{y}_k}^{\text{Pozorované (O)}} - \overbrace{\tilde{\mu}_k}^{\text{Očekávané (E)}}}{\text{Rozptyl}})^2 \sim \chi^2$$

(Formálně zapsáno přes kovarianční matici $\hat{\Sigma}^{-1}$)

Vyhodnocení testu

Rozhodnutí stavíme na výsledné p -hodnotě:

- $p \geq 0,05$: Data jsou MCAR. Chybění je čistě náhodné $\implies$ řádky lze bezpečně smazat.
- $p < 0,05$: Zamítáme MCAR $\implies$ chybění má strukturu (MAR/MNAR). Pouhé smazání by zkreslilo výsledky.

Korelace indikátorů (EDA)

Vytvoření binární matice **M** (1 = chybí, 0 = pozorováno) a vizualizace korelace. Odhalí, zda chybění jednoho sloupce souvisí s chyběním jiného.

Význam hodnoty K v Littleově testu

Co je to p ?

Počet **proměnných** (sloupců) v datasetu. Určuje dimenzi našich dat.

Co je to K ?

Počet **unikátních profilů chybění**, které se v datasetu reálně vyskytují (kde $K \leq 2^p$).

Princip disjunktních skupin:

- Každý respondent spadá **právě do jedné** z K skupin podle toho, co (ne)vyplnil.
- Suma $\sum_{k=1}^K$ sčítá odchylky přes těchto K izolovaných skupin.

Příklad: Dataset se 3 proměnnými ($p = 3$)

Máme proměnné: *Věk*, *Příjem*, *Vzdělání*.

Respondenti vynechávali data 5 různými způsoby:

Skupina k	Profil chybění	Chybí	Vzorek n_k
$(k = 1)$	[ANO, ANO, ANO]	Nic	300
$(k = 2)$	[ANO, NE, ANO]	Příjem	100
$(k = 3)$	[NE, ANO, ANO]	Věk	50
$(k = 4)$	[ANO, ANO, NE]	Vzdělání	30
$(k = 5)$	[NE, NE, ANO]	Věk+Příjem	20

V tomto reálném případě máme v datech:

$$p = 3 \quad \text{a} \quad K = 5 \quad (K > p)$$

w-dcs-missing-Little.ipynb

Listwise Deletion (Odstranění řádků)

- **Mechanismus:** Kompletní vyřazení instance, pokud obsahuje alespoň jedno $M_{ij} = 1$.
- **Riziko:** Pokud data nejsou MCAR, dochází k dramatickému systematickému zkreslení (Bias) odhadů parametrů modelu. Vede k masivní ztrátě statistické síly testů.

Imputace střední hodnotou / Mediánem

- **Mechanismus:** $Y_{ij} \leftarrow \overline{Y}_j$ pro všechna chybějící pozorování.
- Uměle snižuje rozptyl σ^2 veličiny. Likviduje korelační vztahy s ostatními proměnnými.

K-NN Imputace (Geometrický přístup)

Chybějící hodnota je odhadnuta jako vážený průměr k nejbližších sousedů v prostoru $\mathbb{R}^p$.

MICE (Multivariate Imputation by Chained Equations)

Iterativní stochastický proces postavený na sérii podmíněných modelů.

- MICE doplňuje chybějící data tak, že každý sloupec postupně předpovídá pomocí všech ostatních a v opakujících se cyklech tyto odhady vzájemně zpřesňuje, dokud se celá tabulka statisticky neustálí.
- Tento přístup plně zachovává distribuční vlastnosti i korelační vazby proměnných.

Praktická implementace: Naivní přístup (Pandas)

1. Odstranění a konstantní doplňování v Pandas

Vhodné pouze pro ad-hoc analýzy nebo čisté MCAR scénáře (náhodné výpadky).

```
1 df_clean = df.dropna()           # Odstranění řádků s NaN
2 df_cols  = df.dropna(axis=1)     # Odstranění sloupců s NaN
3 df_mean   = df.fillna(df.mean()) # Imputace průměrem CELÉHO sloupce
```

Riziko: Únik dat z testovací množiny (Data Leakage)

Voláním `df.mean()` nad celou tabulkou před jejím rozdělením spočítáme průměr i z dat, která mají zůstat skrytá pro testování. Model se tak nevědomky učí z informací o „budoucnosti“.

Příklad správného postupu (Sklearn):

1. Spočítat průměr **pouze** z trénovacích dat (`.fit()`).
2. Použít tento konkrétní průměr pro trénovací i testovací data (`.transform()`).

2. Modelové a geometrické doplňování (scikit-learn)

Robustní řešení. **Izolace testovacích dat**, brání jejich předčasnému průniku do trénovací fáze.

```
1  from sklearn.impute import SimpleImputer, KNNImputer
2  from sklearn.experimental import enable_iterative_imputer
3  from sklearn.impute import IterativeImputer
4
5  # Jednorozměrná strategie (Průměr / Medián / Nejčastější)
6  imp_simple = SimpleImputer(strategy='median')
7
8  # Vícerozměrná geometrická strategie (K-NN na základě euklidovské vzdálenosti)
9  imp_knn = KNNImputer(n_neighbors=5, weights='distance')
10
11 # Vícerozměrná modelová strategie (MICE - Chained Equations)
12 imp_mice = IterativeImputer(max_iter=10, random_state=42)
```

w-dcs-missing-leakage.ipynb

Praktická implementace imputací v Pythonu - Příklad

Pokročilé doplňování přes nejbližší sousedy ('KNNImputer')

```
1 import numpy as np
2 import pandas as pd
3 from sklearn.impute import KNNImputer
4
5 # Simulace design matice s defektem MCAR
6 X = pd.DataFrame({
7     'Feature_A': [1.2, 2.4, np.nan, 4.8, 5.0],
8     'Feature_B': [5.1, np.nan, 7.3, 8.1, 9.5]
9 })
10
11 # Použití robustního geometrického imputeru (k=2 sousedi)
12 imputer = KNNImputer(n_neighbors=2, weights="distance")
13 X_imputed = pd.DataFrame(imputer.fit_transform(X), columns=X.columns)
14
15 print(X_imputed.round(2))
```

w-dcs-missing-knn.ipynb

Škálování a transformace příznaků

Proč musíme škálovat? (Motivace)

Algoritmy strojového učení neví, co čísla v tabulce znamenají. Nevidí jednotky (roky, Kč, kg), vidí pouze geometrické body v prostoru.

Představme si data o klientech banky:

- **Věk:** Rozsah 18 až 70 let (rozptyl v desítkách).
- **Roční příjem:** Rozsah 300 000 až 2 000 000 Kč (rozptyl v statisících).

Úloha pro algoritmus kNN: Hledáme nejbližšího souseda pro klienta:
 $A = [25 \text{ let}, 500\,000 \text{ Kč}]$.

Máme dva kandidáty:

- $B = [26 \text{ let}, 505\,000 \text{ Kč}]$
- $C = [65 \text{ let}, 500\,000 \text{ Kč}]$

Výpočet vzdálenosti

Algoritmus použije Euklidovskou metriku k výpočtu vzdálenosti od klienta A :

$$\begin{aligned}d(A, B)^2 &= (25 - 26)^2 + (500k - 505k)^2 \\ &= 1 + 25\,000\,000 = \mathbf{25\,000\,001}\end{aligned}$$

$$\begin{aligned}d(A, C)^2 &= (25 - 65)^2 + (500k - 500k)^2 \\ &= 1600 + 0 = \mathbf{1600}\end{aligned}$$

Výsledek špatného měřítka: Podle algoritmu je 65letý senior C **mnohem podobnější** 25letému mladíkovi A než o rok starší člověk B .

Důsledek: Sloupec s většími absolutními čísly (Příjem) zcela vymazal vliv sloupce Věk.

Min-Max škálování (Normalizace)

Nejjednodušším řešením problému různých měřítek je lineární stlačení (nebo roztažení) všech hodnot do fixního uživatelského rozsahu – typicky **[0, 1]**.

Matematický vzorec

Pro každou hodnotu x v daném sloupci spočítáme novou hodnotu x_{new} :

$$x_{\text{new}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Vlastnosti transformace:

- Pokud $x = x_{\min} \implies x_{\text{new}} = 0$.
- Pokud $x = x_{\max} \implies x_{\text{new}} = 1$.
- Absolutně nejmenší hodnota v datasetu se stane nulou, největší jedničkou.

Kdy Min-Max v praxi použít?

- Algoritmy, které striktně vyžadují **pevné hranice** dat (např. neuronové sítě s aktivační funkcí typu Sigmoid nebo Tanh).
- **Zpracování obrazu:** Pixely mají přirozený rozsah 0 až 255. Převod na $[0, 1]$ je standardním krokem před vstupem do CNN.

V scikit-learn pro tuto transformaci najdeme třídu `MinMaxScaler`.

Kritická slabina Min-Max škálování

Upozornění

Min-Max škálování je **extrémně náchylné na přítomnost odlehlých hodnot (outlierů)**. Jediný výkyv v datech může zničit transformaci celého sloupce.

Představme si mzdy 10 zaměstnanců v malém startupu:

- 9 vývojářů má plat v rozmezí **50k až 80k Kč**.
- 1 majitel (outlier) si vyplatí bonus **2 000k Kč** ($x_{\max} = 2000$).

Co se stane po aplikaci Min-Max do rozsahu [0, 1]?

- Majitel dostane hodnotu **1.0**.
- Plat nejhůře placeného vývojáře (50k) se stane hodnotou **0.0**.
- Plat nejlepšího vývojáře (80k) se transformuje na:

$$x_{\text{new}} = \frac{80 - 50}{2000 - 50} = \frac{30}{1950} \approx \mathbf{0,015}$$

Výsledek: Rozdíly mezi platy všech vývojářů (které model potřeboval zkoumat) byly **zhuštěny do miniaturního prostoru mezi 0.0 a 0.015**.

Obecné Min-Max Škálování (Rozsah $[a, b]$)

V praxi často potřebujeme transformovat data do jiného specifického intervalu než pouze $[0, 1]$. Nejčastěji se setkáváme s rozsahem $[-1, 1]$ nebo $[0, 255]$.

Obecný vztah

Chceme-li transformovat původní hodnotu x do nového uživatelského rozsahu $[a, b]$:

$$x_{\text{new}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \cdot (b - a) + a$$

Jak vzorec funguje geometricky?

1. Zlomek nejdříve převede hodnotu do čistého poměru mezi 0 a 1.
2. Násobení $(b - a)$ roztáhne tento poměr na šířku nového intervalu.
3. Přičtení a posune celé mračno dat na novou startovní čáru.

Praktické využití rozsahu $[-1, 1]$:

- **Neuronové sítě:** Pokud skryté vrstvy sítě používají aktivační funkci **Hyperbolický tangens (tanh)**, je nutné, aby data na vstupu byla centrovaná kolem nuly v rozsahu $[-1, 1]$.
- **Zpracování audia / signálů:** Zvukové vlny (PCM data) jsou přirozeně symetrické kolem nuly.

Scikit-learn:

Rozsah definujeme parametrem:

```
MinMaxScaler(feature_range=(-1, 1))
```

Standardizace (Z-score Normalization)

Standardizace transformuje data tak, aby výsledný příznak měl **průměr přesně 0** a **směrodatnou odchylku přesně 1**. Data nemají pevný strop ani dno.

Matematický vzorec (Z-skóre)

Pro každou hodnotu x spočítáme její normalizovanou variantu Z :

$$Z = \frac{x - \mu}{\sigma}$$

Kde μ je průměr a σ směrodatná odchylka sloupce.

Interpretace hodnoty Z :

- $Z = 0 \implies$ Hodnota leží přesně na průměru.
- $Z = 1 \implies$ Hodnota leží 1 směrodatnou odchylku nad průměrem.
- $Z = -2 \implies$ Hodnota leží 2 směrodatné odchylky pod průměrem.

Kdy Standardizaci v praxi použít?

- Je to **výchozí volba** pro většinu algoritmů strojového učení (Lineární a Logistická regrese, SVM, PCA, Lineární diskriminační analýza).
- Mnoho algoritmů předpokládá, že data jsou centrována kolem nuly a mají symetrický rozptyl.

Scikit-learn:

Třída `StandardScaler`.

Odolnost Standardizace vůči outlierům

Vraťme se k našemu startupu s 9 vývojáři (platy 50k až 80k) a 1 majitelem (plat 2 000k). Proč zde Standardizace na rozdíl od Min-Max neselže?

Jak se zachová vzorec $Z = \frac{x-\mu}{\sigma}$?

- Přítomnost obřího platu majitele dramaticky zvedne průměr (μ) i směrodatnou odchylku (σ) celého sloupce.
- Majitelův plat dostane obrovské kladné Z-skóre (např. $Z = +3.1$).
- Platy běžných vývojářů sice spadnou do záporných čísel (protože jsou pod novým, uměle zvýšeným průměrem), ale **jejich vzájemné rozdíly zůstanou zachovány!**

Srovnání

Zatímco Min-Max pošle nejlepšího a nejhoršího vývojáře na hodnoty 0.015 a 0.000 (rozdíl pouhých 0.015), Standardizace je srovná např. na -0.45 a -0.60 (rozdíl 0.15).

Rozdíly mezi běžnými daty jsou zachovány a algoritmus je schopen je dál analyzovat.

Robustní škálování (Robust Scaler)

Co dělat, když data obsahují obrovské množství šumu a extrémních hodnot? Standardní StandardScaler (μ, σ) i MinMaxScaler ($x_{\min}, x_{\max}$) v takové situaci selhávají.

Matematický vzorec

Namísto průměru a směrodatné odchylky použijeme **robustní statistiky**:

$$x_{\text{new}} = \frac{x - \text{median}}{\text{IQR}}$$

Kde $\text{IQR} = Q_3 - Q_1$ je mezikvartilové rozpětí (šířka prostředních 50 % dat).

Proč je to imunní vůči šumu?

- **Medián** (střední hodnota) na rozdíl od průměru μ vůbec nezajímá velikost extrémních konců.
- **IQR** na rozdíl od σ nebo $x_{\max}$ ignoruje

Význam a chování:

- Centrum dat (medián) se posune přesně na **hodnotu 0**.
- Většina běžné populace dat se stabilizuje v kompaktním rozmezí přibližně **od -2 do 2**.
- Případné extrémní outliersy si zachovají svá obrovská čísla (např. hodnota 450), ale **neovlivní transformaci** ostatních běžných prvků.

Scikit-learn:

Třída RobustScaler.

Příklad: Paty startup

Aplikujeme nyní na platy v našem startupu **RobustScaler**:

- 9 vývojářů s platy: 50k, 53k, 55k, 58k, 60k, 62k, 65k, 70k, 80k Kč.
- 1 majitel (outlier): 2 000k Kč.
- Z průzkumové analýzy víme, že: Median = 61k, $Q_1 = 55k$, $Q_3 = 65k \implies \text{IQR} = 10k$.

1. Nejhorší vývojář (50k)

$$x_{\text{new}} = \frac{50 - 61}{10} = -1,1$$

2. Nejlepší vývojář (80k)

$$x_{\text{new}} = \frac{80 - 61}{10} = +1,9$$

3. Majitel / Outlier (2000k)

$$x_{\text{new}} = \frac{2000 - 61}{10} = +193,9$$

Inženýrské zhodnocení výsledku:

- Rozdíl mezi nejlepším a nejhorším vývojářem je po transformaci **rovný 3.0** ($1.9 - (-1.1)$).
- Data běžných zaměstnanců se rozprostřela v čitelném pásmu kolem nuly.
- Majitel odskočil na hodnotu 193.9, ale **neovlivnil** měřítko ostatních.

Jak vybrat správný škálovač?

1. MinMaxScaler

- **Vlastnost:** Stlačí data pevně do rozsahu $[0, 1]$ nebo $[-1, 1]$.
- **Kdy použít:** Obrázky (pixels), neuronové sítě (vstupní vrstvy).
- **Kdy nepoužít:** Data se silnými a nepředvídatelnými outliery.

2. StandardScaler

- **Vlastnost:** Průměr 0, rozptyl 1. Zachovává distribuci dat.
- **Kdy použít:** Lineární modely, PCA, SVM (výchozí bezpečný standard).
- **Kdy nepoužít:** Extrémně asymetrická data s obřími anomáliemi.

3. RobustScaler

- **Vlastnost:** Medián 0, měřítko podle IQR. Odolný vůči šumu.
- **Kdy použít:** Finanční podvody, telemetrie strojů (vysoká pravděpodobnost chyb a anomálií).

Python stack: `.fit()` vs. `.transform()`

Při nasazení škálování (ale i imputace) v reálných projektech musíme striktně oddělit fázi **učení parametrů** a fázi **transformace dat**.

1. Trénovací množina (Train)

Zde provádíme obě operace:

- `.fit()`: Algoritmus se podívá na trénovací data a vypočítá statistiky (μ, σ nebo min, max).
- `.transform()`: Použije zjištěné hodnoty a data fyzicky přeškáluje.

V kódu často spojené do zkratky:

`.fit_transform()`

`w-dcs-scaling.ipynb`

2. Testovací množina (Test)

Zde provádíme **POUZE** transformaci:

- `.transform()`: Data přeškálujeme pomocí statistik z **trénovací** množiny!
- **NIKDY** zde nevoláme `.fit()`.

Proč je `.fit()` na testovací data zakázáno?

- Způsobili bychom **Data Leakage**.
- Model by v testovacích datech viděl budoucnost (např. globální maximum), kterou v produkci mít nebude.
- Výsledky testování by byly falešně optimistické.

Ošetření odlehlých pozorování

Co je to odlehlá hodnota (Outlier)?

Definice

Pozorování, které se tak výrazně odlišuje od ostatních dat, že vzbuzuje podezření, že bylo vygenerováno zcela odlišným mechanismem.

Klíčové inženýrské rozlišení pramenící z praxe:

1. **Chyba / Šum (Bad Data):** Hodnota vznikla selháním lidského faktoru nebo techniky (překlep v dotazníku, zaseknutý plovák nádrže). Tato data **chceme smazat nebo opravit**, protože model pouze poškozují.
2. **Reálná anomálie (Cenný signál):** Data jsou stoprocentně pravdivá, ale reprezentují vzácný, kritický jev. Tato data **nesmíme smazat**. Jsou klíčem k řešení problému.

Příklad cenného signálu: Útok hackerů na server, náhlé selhání leteckého motoru, finanční podvod na kreditní kartě.

V praxi se setkáváme se třemi architektonickými typy odlehlých hodnot:

1. Bodové anomálie (Point Outliers)

Jedna konkrétní instance dat drasticky ulétne mimo jakékoliv běžné rozmezí.

Příklad: Teplotní senzor v serverovně jednorázově nahlásí 150 °C.

2. Kontextové anomálie (Contextual Outliers)

Samotná hodnota je zcela v normálu, ale stává se anomální **vzhledem ke svému kontextu** (čase, poloze).

Příklad: Spotřeba elektřiny v administrativní budově o půlnoci na Štědrý den je stejná jako v úterý v poledne.

3. Kolektivní anomálie (Collective Outliers)

Jednotlivé body samy o sobě nevybočují, ale jejich společná sekvence nebo shluk nedává statisticky smysl.

Příklad: CPU serveru vykazuje běžnou zátěž, ale graf tvoří dokonale rovnou čáru bez jakéhokoliv přirozeného šumu po dobu 3 hodin.

Jednorozměrná detekce (Univariate Outliers - To už umíme)

Pokud analyzujeme každý sloupec tabulky izolovaně, používáme dva standardní statistické pilíře:

1. Z-skóre a pravidlo tří sigma (3σ)

Vhodné pro data s přibližně **normálním (Gaussovým) rozdělením**.

$$Z_i = \frac{y_i - \bar{y}}{\sigma}$$

Pokud $|Z_i| > 3$, pozorování leží dál než 3 směrodatné odchylky od průměru a je označeno za outlier (mimo 99,73 % populace).

2. Metoda IQR (Interquartile Range)

Robustní metoda nezávislá na rozdělení dat (odolná vůči extrémním hodnotám). Používá se v Boxplotech.

$$\text{IQR} = Q_3 - Q_1$$

Hranice pro odlehlé hodnoty:

$$\text{Limity} = [Q_1 - 1,5 \cdot \text{IQR}, Q_3 + 1,5 \cdot \text{IQR}]$$

Cokoliv leží mimo tyto „vousy“ grafu, je jednorozměrný outlier.

Limity jednorozměrné detekce: Proč ve 2D+ selhává?

Problém: Maskování (Masking)

Jednorozměrné metody jsou **zcela slepé vůči korelační struktuře dat**. Nedokážou odhalit anomálii, která vznikne kombinací hodnot.

Představme si data z monitoringu motoru: [Otáčky] a [Teplota]

- **Otáčky:** Běžný rozsah je 1000 až 6000 RPM. Hodnota 1500 RPM je úplně v normě.
- **Teplota:** Běžný rozsah je 40 až 110 °C. Hodnota 105 °C je úplně v normě.

Kde nastává selhání? Pokud motor běží na pouhých 1500 **RPM** (skoro volnoběh), ale teplota je 105 °C, systém je v kritické anomálii (pravděpodobně selhalo chlazení).

- Z-score ani IQR sloupce [Otáčky] nic nepozná (1500 je v normě).
- Z-score ani IQR sloupce [Teplota] nic nepozná (105 je v normě).

Izolovaně jsou obě hodnoty perfektní, ale jejich **společný výskyt je kritická anomálie**.

Geometrický pohled: Proč Euklidovská vzdálenost selhává?

Mějme vícerozměrný prostor $\mathbb{R}^P$. Nejintuitivnější metrika je Euklidovská vzdálenost d_E .

Matematický zápis

Vzdálenost bodu $\mathbf{x}_i$ od středu (průměru μ):

$$d_E^2 = (\mathbf{x}_i - \mu)^T (\mathbf{x}_i - \mu)$$

Kritický předpoklad: Euklidovská metrika předpokládá, že všechny proměnné jsou **nezávislé** a mají **stejný rozptyl** (shluk dat tvoří v prostoru dokonalou, symetrickou kouli).

Selhání v praxi

Pokud proměnné korelují (např. protažená elipsa vztahu Otáčky $\rightarrow$ Teplota), Euklidovská vzdálenost měří špatně.

- Body ležící na dlouhé ose elipsy (přírozená variabilita) považuje za **příliš vzdálené** (falešné anomálie).
- Body ležící kolmo mimo elipsu (skutečné anomálie) považuje za **blízké** (nedokáže je odhalit).

Vzdálenost: Pohled ze střední školy

Matematické okénko

Otázka

Proč v datové vědě píšeme $d_E^2 = (\mathbf{x}_i - \boldsymbol{\mu})^T (\mathbf{x}_i - \boldsymbol{\mu})$ namísto klasického středoškolského vzorce?

1. Klasická geometrie (Skaláry)

Představme si bod o souřadnicích $[x_1, x_2]$ a střed $[\mu_1, \mu_2]$. Jejich vzdálenost v rovině sčítáme po jednotlivých složkách (osa X a osa Y):

$$d_E^2 = (x_1 - \mu_1)^2 + (x_2 - \mu_2)^2$$

Kritický inženýrský problém v praxi:

- Pokud máme tabulku s 50 příznaky (dimenzemi), vzorec by měl 50 členů.
- Programovat takový výpočet v Pythonu nativně přes cyklus `for` řádek po řádku je pro velké datasety extrémně pomalé.

2. Moderní datová věda (Lineární algebra)

Celý i -tý objekt (řádek tabulky) reprezentujeme jako jeden **sloupcový vektor** $\mathbf{x}_i$. Rozdíl od středu $\mathbf{v}$ je pak vektor jeho vnitřních složek:

$$\mathbf{v} = (\mathbf{x}_i - \boldsymbol{\mu}) = \begin{pmatrix} x_1 - \mu_1 \\ x_2 - \mu_2 \end{pmatrix}$$

Pozor na indexy: $\mathbf{x}_i$ značí celý i -tý objekt, zatímco x_1, x_2 jsou sloupce (příznaky).

Podle pravidel maticového násobení nemůžeme vynásobit dvě stojící nudle tvaru $(2 \times 1) \cdot (2 \times 1)$. Proto první vektor **transponujeme** (T), čímž ho položíme na bok:

$$\mathbf{v}^T \cdot \mathbf{v} = \begin{pmatrix} v_1 & v_2 \end{pmatrix} \cdot \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} = (v_1 \cdot v_1) + (v_2 \cdot v_2) = v_1^2 + v_2^2$$

Výpočetní benefit: Moderní procesory (instrukce SIMD/AVX) umí operaci $\mathbf{v}^T \mathbf{v}$ spočítat hardwarově naráz v jednom kroku, ať už má vektor 2 nebo 5000 dimenzí.

Řešení: Mahalanobisova vzdálenost (D_M)

Abychom viděli anomálie správně, musíme metriku naučit respektovat směr a tvar datového shluku.

Matematická definice

Do vzorce vložíme inverzní **kovarianční matici** Σ^{-1} :

$$D_M^2 = (\mathbf{x}_i - \mu)^T \Sigma^{-1} (\mathbf{x}_i - \mu)$$

Jak to funguje geometricky?

1. Σ^{-1} **odstraní korelaci** (otočí prostor podle hlavních os dat).
2. **Normalizuje rozptyl** (připodobní elipsu na dokonalou kružnici).

Inženýrský význam

Mahalanobisova vzdálenost měří vzdálenost v počtech **směrodatných odchylek v daném směru**, nikoliv v milimetrech.

Vlastnosti pro praxi:

- Hranice konstantní vzdálenosti D_M netvoří kruh, ale dokonale kopírují **tvar elipsy** našich dat.
- Bod na okraji elipsy ve směru trendu má malé D_M . Bod stejně daleko, ale kolmo mimo trend, má obrovské $D_M \implies$ **úspěšná detekce**.

Matematické okénko: Anatomie kovarianční matice

Kovarianční matice Σ (Sigma) je čtvercová matice typu $(p \times p)$, kde p je počet příznaků (sloupců). Reprezentuje celkovou geometrickou strukturu datového mračna.

Představme si prostor 2D (Otáčky x_1 a Teplota x_2):

$$\Sigma = \begin{pmatrix} \sigma_1^2 & \sigma_{12} \\ \sigma_{21} & \sigma_2^2 \end{pmatrix}$$

Co znamenají jednotlivé prvky uvnitř?

- **Hlavní diagonála** (σ_1^2, σ_2^2) : Obsahuje **rozptyly** jednotlivých veličin. Říká, jak moc jsou data protažená v samostatných osách X a Y .
- **Mimo diagonálu** $(\sigma_{12} = \sigma_{21})$: Obsahuje **kovarianci** mezi oběma veličinami. Říká, zda veličiny korelují (rostou společně).

Tři geometrické scénáře pro praxi:

1. $\sigma_{12} = 0 \implies$ Data tvoří **kruh** nebo elipsu srovnanou podle os (proměnné jsou nezávislé).
2. $\sigma_{12} > 0 \implies$ Data tvoří **nakloněnou elipsu** (s rostoucími otáčkami roste teplota).

Matematické okénko: Aplikace inverzní matice Σ^{-1}

Vzpomeňme si na vzorec: $D_M^2 = (\mathbf{x}_i - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x}_i - \boldsymbol{\mu})$. Proč je tam ta matice „vzhůru nohama“?

1. Analogie s jednorozměrným Z-skóre V 1D prostoru dělíme směrodatnou odchylkou (normalizujeme rozptyl):

$$Z^2 = \frac{(x - \mu)^2}{\sigma^2} = (x - \mu) \cdot \frac{1}{\sigma^2} \cdot (x - \mu)$$

Vícerozměrná inverzní matice $\boldsymbol{\Sigma}^{-1}$ je přímým ekvivalentem tohoto dělení $\frac{1}{\sigma^2}$.

2. Co přesně $\boldsymbol{\Sigma}^{-1}$ s prostorem udělá?

- **Zruší korelaci (Dekorelace):** Otočí souřadný systém tak, aby osy běžely přesně podél hlavního směru protažení datové elipsy.
- **Zruší měřítko (Zbělení / Whitening):** Stlačí protažené směry a roztáhne ty krátké.

Výsledek: Z pohledu inverzní matice se jakkoliv deformovaná a nakloněná datová elipsa promění zpět na **dokonalou, symetrickou kružnici** (kouli). V tomto „narovnaném“ prostoru už můžeme bezpečně měřit vzdálenost.

Redukce dimenzionality a lineární transformace

Prokletí dimenzionality (Curse of Dimensionality)

Paradox datové vědy

Větší množství dat obvykle znamená lepší model. To ale platí pro **počet řádků** (objektů), nikoliv pro nekontrolovaný **počet sloupců** (příznaků).

Geometrický problém: Exponenciální růst prostoru

S rostoucím počtem rozměrů roste objem prostoru tak rychle, že se v něm data doslova ztratí:

- **1D prostor (Úsečka):** Máme úsečku délky 1 metr. Pokud na ni náhodně hodíme 10 bodů, budou relativně blízko sebe.
- **2D prostor (Čtverec):** Máme plochu 1×1 metr. Stejných 10 bodů se v ní už docela ztratí.
- **3D prostor (Krychle):** V kostce $1 \times 1 \times 1$ metr jsou tyto body od sebe již velmi izolované.
- **100D prostor (Hyperkrychle):** Prostor je tak nepředstavitelně obrovský, že body jsou od sebe daleko – prostor je „pustý“.

Důsledek: V pustém prostoru ztrácejí pojmy jako „vzdálenost“ nebo „nejbližší soused“ svůj smysl. K zaplnění vysokorozměrného prostoru bychom potřebovali astronomické množství řádků, jinak se model okamžitě přeučí (*Overfitting*).

Problém multikolinearity

Co je to multikolinearita?

Nastává v situaci, kdy jsou dva nebo více sloupců v datové tabulce **silně korelované** – jinými slovy, přenášejí do modelu tutéž informaci.

Typické příklady z praxe:

- *Duplicita jednotek*: Výška v centimetrech a Výška v palcích.
- *Fyzikální provázanost*: Teplota motoru, teplota oleje a teplota chladicí kapaliny.

Proč to algoritmům strojového učení vadí?

1. **Matematická nestabilita**: Pokročilé algoritmy (např. lineární regrese) provádějí inverzi matic. Pokud jsou sloupce závislé, determinant matice se blíží nule a výpočet generuje velké zaokrouhlovací chyby nebo zcela selže.
2. **Zmatení modelu**: Pokud dva parametry rostou lineárně spolu, model nedokáže matematicky rozklíčovat, který z nich má na cílovou veličinu reálný kauzální vliv. Výsledné váhy modelu pak divoce skáčou.

Jak z toho ven? Náznak řešení pomocí PCA

Detailně si algoritmus vysvětlíme u nesupervizovaného učení. Nyní nám stačí **intuice**, jak vyřešit multikolinearitu a snížit dimenzi.

Geometrická myšlenka PCA (Principal Component Analysis): Představte si data ve 2D (např. *Teplota motoru* a *Teplota oleje*), která spolu silně korelují a tvoří v grafu protáhlé mračno.

- **Otočení souřadného systému:** PCA vezme původní osy a otočí je tak, aby nová hlavní osa (**PC1 – První hlavní komponenta**) vedla přesně ve směru **největšího rozptylu** (skrz nejdelší část mračna).
- **Zachycení informace:** Osa PC1 v sobě koncentruje maximum informace z obou původních sloupců.
- **Nekorelovanost:** Druhá osa (**PC2**) je kolmá na tu první a zachycuje zbytek šumu. Tyto nové osy jsou na sebe **naprosto nezávislé** (korelace = 0).

Výsledek

Místo dvou korelovaných sloupců ponecháme pouze jeden jediný sloupec: **PC1**.
Vyřešili jsme multikolinearitu a zredukovali dimenzi dat (z 2D do 1D), přičemž jsme nepřišli o podstatu informace.

Redukce početnosti: Teorie vzorkování

Redukce početnosti: Proč a jak vzorkovat?

Je-li je dataset rozsáhlý, že se nevejde do paměti (RAM) nebo trénování modelu trvá dny?
Řešením je **vzorkování (Sampling)** – výběr reprezentativní podmnožiny.

Základní cíl vzorkování

Chceme vybrat vzorek dat n z celkové populace N takovým způsobem, aby si tento vzorek **zachoval statistické vlastnosti původního celku** (stejný průměr, rozptyl, rozdělení).

Základní typy pravděpodobnostního vzorkování:

- **Prostý náhodný výběr (Simple Random Sampling - SRS):** Každý řádek v datasetu má naprosto stejnou pravděpodobnost, že bude vybrán.
- **Stratifikovaný výběr (Stratified Sampling):** Dataset nejdříve rozdělíme do skupin (strat) – např. podle pohlaví nebo věku – a z každé skupiny navzorkujeme data samostatně. Zajistíme tím, že minoritní skupiny v datech nezmizí.

Mechanika náhodného výběru: S vracením vs. Bez vracení

Při realizaci prostého náhodného výběru (SRS - Simple Random Sampling) musíme rozhodnout, zda vybraný řádek můžeme vylosovat znovu, či nikoliv.

1. SRSWR

SRS With Replacement

(Náhodný výběr s vracením).

- Každý vybraný řádek vrátíme zpět do osudí.
- Pravděpodobnost výběru zůstává pro všechny kroky **konstantní** ($1/N$).
- Jeden konkrétní řádek se může ve finálním vzorku objevit **vícekrát**.

Využití: Základ pro pokročilé algoritmy jako Bagging a **Random Forest** (metoda Bootstrapping).

2. SRSWOR

SRS Without Replacement

(Náhodný výběr bez vracení).

- Jakmile řádek vylosujeme, vyřadíme jej.
- Pravděpodobnost výběru se s každým dalším krokem **mění** ($1/N, 1/(N-1) \dots$).
- Každý řádek se ve výsledném vzorku objeví **maximálně jednou**.

Využití: Standardní rozdělení dat na trénovací a testovací množinu (`train_test_split` v `scikit-learn`).

Stratifikovaný výběr (Stratified Sampling)

Co když náhodný výběr (SRS) selže? Pokud pracujeme s **nevyváženými daty**, prostá náhoda může z finálního vzorku zcela vymazat klíčovou minoritní skupinu.

Praktický příklad: Analýza odchodu zákazníků (Churn)

- 9 900 klientů je spokojených a pouze 100 klientů odešlo (pouhé 1 % dat), celkem 10 000.
- Chceme vytvořit trénovací vzorek o velikosti $n = 500$ řádků.

Riziko prostého výběru (SRS)

Pokud spustíme klasický náhodný výběr, může se snadno stát, že mezi 500 vylosovaných klientů **netrefíme ani jednoho**, který odešel.

Následek: Model se nenaučí anomálii rozpoznat, protože ji ve vzorku vůbec neuvidí.

Řešení: Stratifikace

Data rozdělíme do skupin (**straty**) podle cílové třídy.

Vzorkujeme z každé straty samostatně tak, aby byl **zachován poměr** 99 : 1:

- Vylosujeme 495 spokojených.
- Vylosujeme přesně 5 těch, co odešli.

Shrnutí: Stratifikovaný výběr garantuje, že každá statistická vrstva (stratum) bude ve výsledném vzorku zastoupena **ve stejném poměru jako v původní populaci**.

Transformace a inženýrství příznaků

Kódování kategoriálních proměnných

Většina algoritmů strojového učení (např. lineární modely, neuronové sítě) neumí pracovat s textem. Všechny kategorie musíme převést na čísla.

1. Label / Ordinal Encoding

Přiřadí každé kategorii unikátní celé číslo (např. *Malý* $\rightarrow$ 0, *Střední* $\rightarrow$ 1, *Velký* $\rightarrow$ 2).

- **Kdy použít:** Pokud má kategorie přirozené uspořádání (ordinální data).
- **Riziko:** U nominálních dat (*Praha*=0, *Brno*=1, *Ostrava*=2) si model může vymyslet falešný vztah, že $2 > 0$ (Ostrava je víc než Praha).

2. One-Hot Encoding

Pro každou kategorii vytvoří nový binární sloupec (0 nebo 1).

- **Kdy použít:** Pro nominální data bez jasného pořadí (Město, Barva, Pohlaví).
- **Výhoda:** Žádná kategorie není matematicky „nadřazená“ jiné.
- **Riziko:** Pokud má sloupec tisíce unikátních hodnot (vysoká kardinalita), extrémně to nafoukne šířku tabulky.

Diskretizace (Binning) a Vyhlazování dat

Co je to Binning?

Proces převodu **spojité proměnné na diskrétní kategorie** (příhrádky). Pomáhá zjednodušit model, stabilizovat výpočty a snížit vliv outlierů.

Dva základní přístupy k tvorbě příhrádek:

1. **Stejná šířka (Equal-Width Binning)**: Rozsah od minima po maximum rozdělíme na k stejně velkých intervalů. (Např. věk: 0–30, 30–60, 60–90).
 - *Problém*: Pokud jsou v datech outlieri, některé příhrádky zůstanou prázdné.
2. **Stejná frekvence / Kvantily (Equal-Frequency Binning)**: Intervaly nastavíme tak, aby v každé příhradce byl **přesně stejný počet řádků**.

Vyhlazování dat (Data Smoothing)

Pokud data obsahují lokální šum, binning nám umožní data vyhladit. Všechny hodnoty uvnitř příhrádky nahradíme buď **průměrem**, nebo **mediánem** dané příhrádky.

Tvorba nových příznaků (Feature Construction)

Klíč k úspěchu v Data Science

Surová data málokdy obsahují vztahy přímo. Často musíme zapojit **doménovou znalost** a vytvořit nové sloupce kombinací těch stávajících.

Typické techniky v inženýrské praxi:

- **Matematické kombinace (Ratios & Interakce):**

- Máme sloupce *Váha* a *Výška* → složíme je do jednoho: $BMI = \frac{Váha}{Výška^2}$
- Máme *Celková cena nákupů* a *Počet nákupů* → *Průměrná hodnota objednávky*.

- **Dekompozice času (Datetime Splitting):** Modely neumí pracovat s textem "2026-05-25 08:00:00". Rozložíme ho na sloupce: *Hodina* (0–23), *Den v týdnu* (0–6), nebo binární flag *Je_Víkend*.

- **Logické indikátory (Flags):** Převod složité logiky na 0/1 (např. sloupec *Je_Student* na základě věku a statutu).

w-dsc-transform.ipynb

- 1 Přechod od diagnostiky k sanaci dat
- 2 Taxonomie datové kontaminace (dirty data)
- 3 Teorie chybějících hodnot
- 4 Škálování a transformace příznaků
- 5 Ošetření odlehlých pozorování
- 6 Redukce dimenzionality a lineární transformace
- 7 Redukce početnosti: Teorie vzorkování
- 8 Transformace a inženýrství příznaků